Last updated: 20th September 2026

IS212 (AY 2026-27, T1) – Week 6: Automated Testing Assignment

Background



You have been hired as a Software Engineer in Test by **Air Britannia**, a long-established airline flying from London to destinations across Europe, Asia and the Americas.

Air Britannia has developed a Python application that calculates the miles earned by members of its **Britannia Club** loyalty programme. Every sector flown earns two separate figures: **redeemable miles**, which members spend on reward flights, and **elite miles**, which count towards the member's tier for the following year. The two figures are not the same, and some sectors earn nothing at all.

The accrual rules are described below, and are applied by the miles calculator in sequence:

1. **Minimum Sector Rule:** a sector is credited with the actual distance flown, in miles. However, a sector shorter than 500 miles is credited as 500 miles. This applies to both figures.
2. **Fare Class Rule:** both figures are multiplied by an earning percentage determined by the fare class booked: First 200%, Business 150%, Premium Economy 110%, Economy Flex 100%, Economy Standard 75%, Economy Lite 25%, and Economy Saver 0%.
3. **Partner Airline Rule:** if the sector is operated by a partner airline rather than by Air Britannia, both figures are reduced by 50%. In addition, Economy Lite and Economy Saver fares earn nothing at all when flown on a partner.
4. **Tier Bonus:** a bonus is added according to the member's current tier: Blue 0%, Silver 25%, Gold 50%, and Solitaire 100%. This bonus applies to the redeemable miles only – elite miles are never increased by it.
5. **Milestone Bonus:** if the sector takes the member's elite mile balance for the membership year from below 25,000 to 25,000 or above, a one-off bonus of 5,000 redeemable miles is awarded on that first crossing.
6. **Rounding Rule:** both figures are always rounded down to whole miles.

For instance, if Chris (a Gold member) flies from London to Los Angeles (5,400 miles) in Business on an Air Britannia aircraft, he will earn ((5,400 × 150%) × 150%) = 12,150 redeemable miles and (5,400 × 150%) = 8,100 elite miles.

If Ada (a Blue member) flies from London to Edinburgh (330 miles) in Economy Standard, the sector is credited as 500 miles because it falls below the minimum, so she earns (500 × 75%) = 375 redeemable miles and 375 elite miles.

Finally, suppose Boudicca (a Silver member) has already earned 22,000 elite miles this membership year. She flies from London to Boston (3,200 miles) in Economy Lite on a partner airline, which earns her nothing on either count. She then flies from London to Singapore (6,800 miles) in Business on an Air Britannia aircraft, earning (6,800 × 150%) = 10,200 elite miles – which takes her past the milestone – and ((6,800 × 150%) × 125%) + 5,000 = 17,750 redeemable miles.

Tasks

The goal of this assignment is to **produce unit tests** for the Britannia Club miles calculator that:

1. Achieve 100% statement coverage of accrual.py
2. Are able to detect injected faults

We have provided an implementation of the business logic in **accrual.py**, as well as a few example accrual calculations in **examples.py**. Note that you must not alter these files, and you can assume them to be correct. **Altering the implementation in any way will lead to a score of 0 marks.**

We have also provided **unit_tests.py**, which contains a blank testing class, "TestMilesCalculator". Your task is to add code within TestMilesCalculator only to comprehensively test accrual.py using "unittest". You may add test methods, setUp()/tearDown(), and helper methods within this class.

Everything in **accrual.py** is in scope for testing, including input validation and updates to a member's balances. Your tests should aim for 100% statement coverage (as measured by the "coverage" tool) and should check both redeemable miles and elite miles. Where relevant, they should also check the resulting balances stored in the Member object. You may test MilesCalculator and/or the individual rule classes; grading is based on coverage and fault detection rather than a prescribed test structure. Where the rules above differ from how real airline loyalty programmes work, the rules above are the ones your tests must follow.

Submission & Deadline

You are to individually submit your updated version of **unit_tests.py** only.

You must *not* submit any other file.

Furthermore, in unit_tests.py, you should only add code within the TestMilesCalculator class. You may remove the existing pass statement, but do not modify any code outside the class. This assessment is **auto-graded**, and modifying the code in any other way **will lead to 0 marks**.

This assignment is worth 2.5% of the overall mark for IS212. Once you are satisfied with your unit tests, please submit **unit_tests.py** only to eLearn **by 23:59 on Friday 2nd October**. There will be a **penalty of 50% for late submission** up to 24 hours after the deadline. No submissions will be accepted beyond this point.

Grading (2.5 marks)

Important: before submitting, execute "python unit_tests.py" in the same folder as the original, unaltered version of accrual.py. All your tests must pass, otherwise your **overall mark will be 0**.

Assuming your tests pass, **1 mark** will be awarded for the **statement coverage** of accrual.py achieved by your unit tests, as measured by the "coverage" tool introduced in the Week 5 lecture slides and lab (please refer to those resources for instructions how on measure coverage). 100% coverage will lead to 1 mark; the mark will be reduced proportionately if the code coverage is less than that.

To calculate the remaining **1.5 marks**, we will run your unit tests against several variants of accrual.py in which **faults have been injected** (e.g. to the business logic). Your mark for this component will reflect the proportion of faults that your unit tests were able to detect. The faulty code will not be provided before submission but will be provided with your marks.

You are allowed to use AI tools in this assessment (e.g. for generating unit tests), but you are accountable for ensuring your test suite satisfies the requirements above and for ensuring it is able to detect faults.

Having said that, try to complete the exercise by hand *first* before using AI to improve your solution. It is a good opportunity to strengthen your testing fundamentals. Understanding and fixing unit tests is in-scope for the quizzes and exam.